Software Engineering.

→ Kind of engineering in which Software is directed for computer system or any electronic devices

→ Father → Barry Boehm

* Characteristic:-

1) Large Software
2) Cost
3) Dynamic Nature.
4) Quality management.
5) Scability.

* Software Testing:-

→ The process of executing Software whose main objective is to find bugs.

1) Unit Testing:-

→ known as Component Testing / module testing / function Testing.
→ Done during Coding phase.
→ By programmer.

2) Integration Testing:-

→ Testing by combining all units/component together.
→ To see all unit function together.
4 types:- (a) Top-bottom (b) Bottom up.
(c) Sandwich - (Top down + bottom up) hybrid.
(d) Big-bang:- All units are combined in one group and tested together. Best for small system.

(3) Alpha testing:-

→ Done before releasing software to users, to make it error free and issue-free.
→ Environment used :- Developer Environment.
→ Requires coding knowledge.
→ Performed by Sofware Engineer.

(4) Beta testing:-

→ Testing done by Customers.

(5) Stress testing:-

→ Used to verify the stability and reliability of software.
→ check how much load the software can bear and after load it will crash.
→ Non-functional testing
→ known as Endurance testing or Torture Testing.

(6) Recovery Testing:-

→ Validated whether, when the software crashed, it can recover or not.
→ Performed by testing team.

(7) Security testing:-

→ Testing check whether the Software can keep data secure or not
→ check how data security is maintained if the Software is hacked
→ non-functional testing

(8) Smoke Testing:-

→ Build verification Testing.

→ When the development team provides a new build (version), the Software Testing validates the build and checks whether it is error-free.

(9) Regression Testing:-

→ To ensure that changes made in the Software are not causing any new issues in its working.

→ It is done repeatedly.

(10) System Testing:-

→ whether the development Software works on as many operating Systems as required or not.

→ Type of black box testing.

Aspect	Alpha	Beta
1) Objective	Identify internal errors & validate core functionalities	Gather real world feedback & validate usability
2) Participants	Software Developers & Quality Assurance Team	External users (limited)
3) Focus	Functionality, Performance, Security, ~~Before beta testing~~	User Experience, Compatibility ~~before final release~~
4) Timing	Before beta	before final release
5) method	White & black box testing	Black box & Exploratory testing
6) Risk	Low risk and controlled	High risk due to large audience

(11) Performance testing :-

→ Done by Performance Engineer.
→ Used to test runtime performance.
→ Check Software Speed

(12) Ad-hoc testing :-

→ Performed without any planning or documentation.

Aspects	Black	White	Grey
Knowledge of code	No knowledge	Full knowledge	Partial knowledge
Focus	Functionality Behaviour	Internal code & logic	Combination of both
Performed by	Tester, Quality Assurance Engg.	Developer	Testers with some knowledge
Testing approach	Input-output validation	Code path Analysis	Both functional & structured testing

* SDLC (Software development life cycle) :-

(1) Requirement and Analysis :-

→ Details are gathered about what we want the Software to achieve.

(2) Feasibility Study :-

→ A detailed Study of the requirements is done and the possibility of the Software projects success is checked.
→ Cost Analysis of the project is also done.

→ The objective is not to solve the problem, but to decide whether the problem can be solved or not.

Types: (a) Technical, (b) ~~[illegible]~~ operational, (c) Economical feasibility

(3) Design :-
→ It is the blueprint of system/software.
→ The system is designed using the complete technical specification

(4) Coding :-
→ Coding work.
→ The real software is started in this phase

(5) Testing :-
→ Code verification & validation done in phase.
→ It is a long-lasting phase.

(6) Deployment :-
→ Deployment of the completed software/system at the end user.

(7) Maintainance :-
→ long-lasting process.

Types: (a) Corrective (b) Adaptive (c) Perfective (d) Preventive

Verification	validation
• Performed by QA Team	• Executed by testing team.
• To check apps which is according to specification	• Focus on validating the actual product
• Right product.	• Product is right
• ~~Check~~ Check product.	• Check functionality.
• White Box testing	• Black box testing.

* SRS (Software Requirement Specification):-

→ A document/description of all requirements needed to develop software.
→ What feature it will have.
→ Acts as a contract between the client and the development team.
→ Reduces the time taken in the SDLC.
→ The client can use the SRS to check whether the product matches the requirement or not.

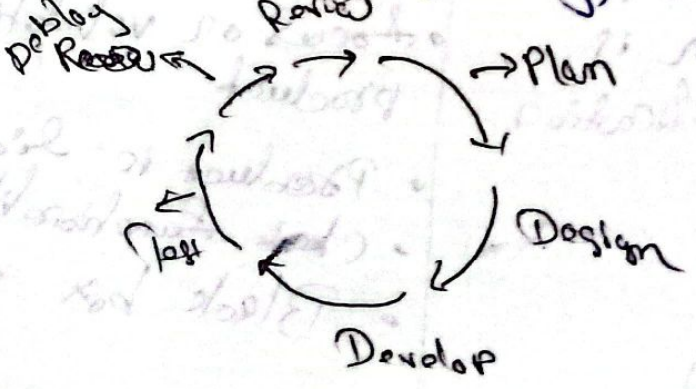
* Software Development models:-

(1) Agile:-

→ It is combination of iterative and incremental development
→ Main focus on process adaptability and customer satisfaction
→ All iterations/modules are kept small.

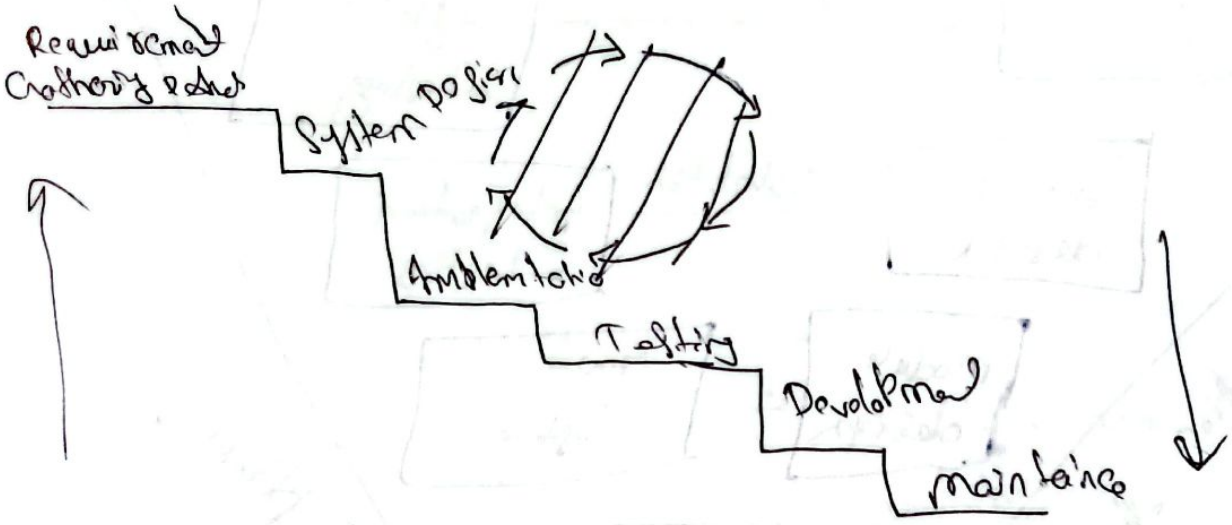
* Principles:-

→ Customer representative who reviews and evaluates the requirement after every iteration
→ Not rely on documentation, Demo of the working software
→ Usually two programmers work together. one writes code other reviews. (Pair programming)



(2) Waterfall model:-

→ linear and sequential model
→ A development phase does not start until the previous phase has finished.
→ Phases cannot overlap.
→ once a phase is completed and you move to the next phase, you cannot go back to previous phase.
→ The output of one phase become the input of the next phase.

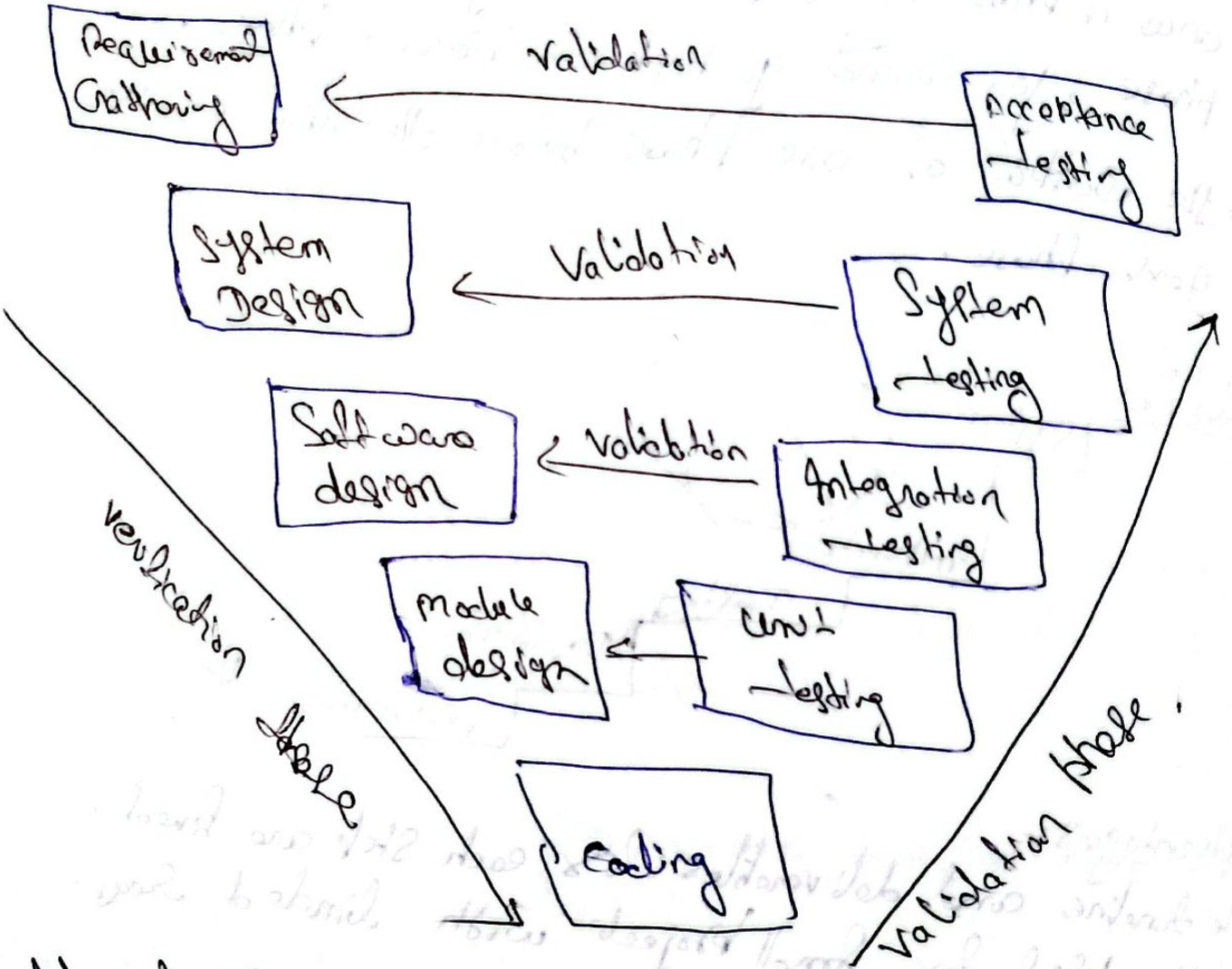


* Advantages:-
→ The timeline and deliverables for each step are fixed.
→ mostly used for small projects with limited resources.

* Disadvantages.
→ lack of flexibility
→ less user involvement.
→ No risk management

(3) V-Model :-

- → verification and validation model
- → Extention of the Waterfall model
- → Shape like 'V' pattern, Development and testing run in parallel.



* Advantages :-

→ Dedicated testing step for every development step.

→ Strong emphasis on documentation

→ Because testing is done in every phase, its quality and reliability are better.

* Disadvantages:-

→ Not flexible
→ Not useful for large and complex projects.

(4) Incremental model:-

→ The whole system is not built at once.
→ Developed in seperate increments
→ Each increment, the new part is added to the already developed software.

* Advantages:-

→ Easy testing, less risk
→ Easy to change according to the client's needs.

* Disadvantages:-

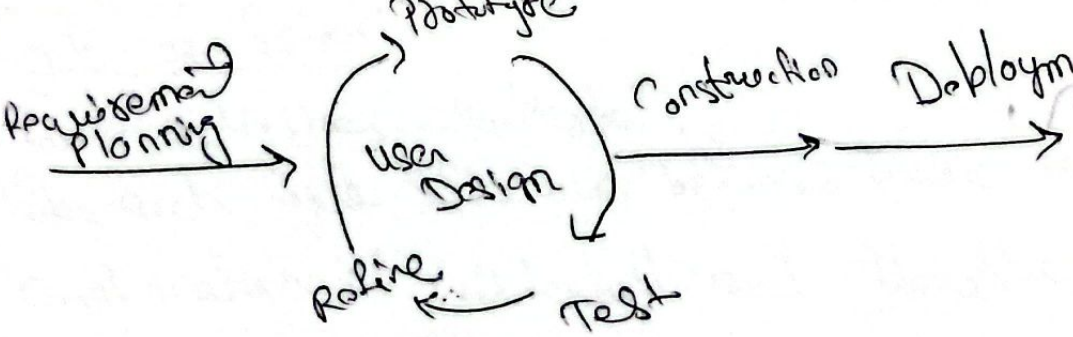
→ Requires very good design and planning.
→ Complexities can arise.

(5) RAD Model (Rapid Application Development)

→ Understand user's requirements. Quickly and develop the software rapidly. Base on prototypes.

* Phases:-

i) Requirement planning phase.
ii) User Design phase
iii) Constructive phase
iv) Deployment phase



* Advantages:
→ Development takes less time.
→ High Quality Software is built Quickly.
→ Repeated feedback makes the Software better.

* Disadvantages:-
→ Difficult for less-experience developers
→ Useful only for small and medium-Size projects.

(6) Iterative model:-
→ The Software is developed in small parts.
→ Each Step includes testing and review.
→ The Software is not built all at once.

* Characteristics:
i) Phased development.
ii) Customer feedback.
iii) Reduced risk.

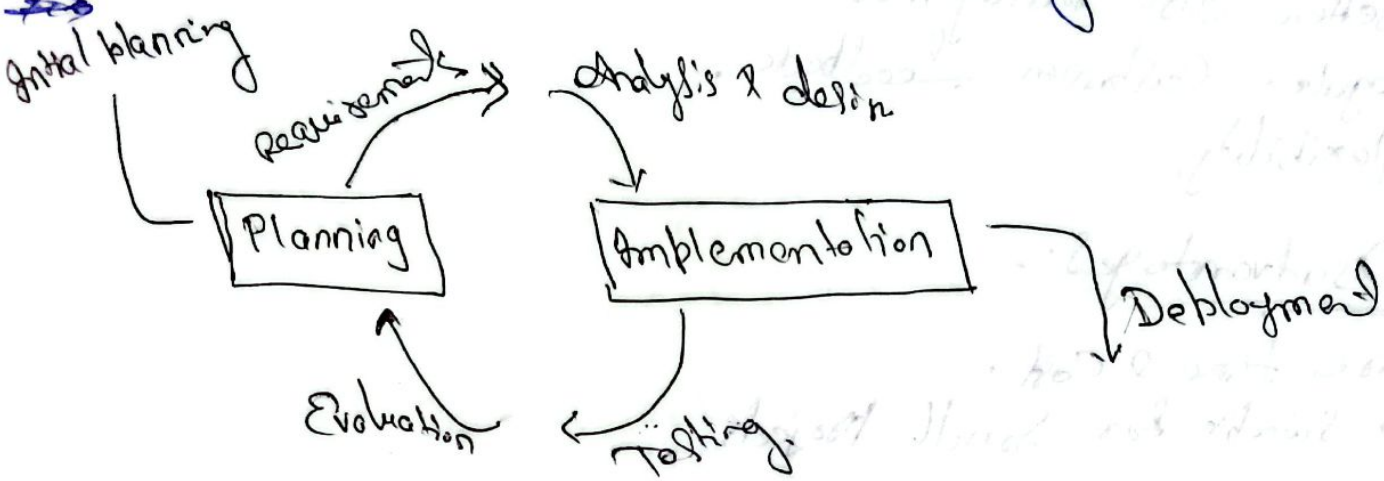
* Phases:-
i) Requirement gathering
ii) Design
iii) Implementation
iv) Testing.
v) Evaluation.

* Advantages:-

→ Working Software is obtained Quickly.
→ Easy to develop large Software in small parts.

* Disadvantages:-

→ Time and cost increase because of many iterations



(7) Spiral model:-

→ Reduced project risks
→ Called Spiral model because development proceeds in Regular interations (cycle)

* Four major activities:-

(a) Determine objective and Alternatives
(b) Risk Analysis.
(c) Development and testing
(d) Evaluation & Planning.

* Cycle of Spiral:-

→ four activities repeated.
→ After each cycle Software becomes more mature.
→ Combination of waterfall and Iterative model.

* Risk management.
* Interactive and iterative development.
* ~~System~~ Suitability

* Advantages:-

→ Better risk management.
→ Regular Customer feedback.
→ flexibility.

* Disadvantages:-

→ more time & cost.
→ Not suitable for small projects.

(8) Prototype Model:-

→ Used when the Customer's requirements are not clearly defined
→ A working model is built, so that the customer can see how it works and suggest necessary changes.

* Process Step:-

Requirement gathering.
Prepare Quick design.
Build working prototype.
User Evaluation & feedback.
Refinement and improvement in prototype.

* Advantages:-

→ Requirement can be understood clearly.

* disadvantages:-

→ Not suitable for large and complex project.

(9) Cocomo model:-

→ Constructive Cost Model.

* Types:

(a) Basic Cocomo model:

→ Simplest, Depends only on Software size (KLOC). Thousand line of code.

3 categories:

(i) Organic – Small and easy.

(ii) Semi-detected – moderate

(iii) Embedded – Complex & hard Product.

(b) Intermidiate Cocomo model.

(c) Detailed Cocomo model.

(10) Big-bang model:

→ Coding start without proper planning and without fully understanding of Customer's requirement.

→ Developed, Delivered once, No testing.

* Advantage:

→ Suitable for small project.

* Disadvantage:

→ High Risk, No documentation.

(11) Kanban model:-

→ Started by Toyota company.

→ used to manage work flow.

* Size estimation in Software project :-

(A) Size measurement :-

(1) LOC (Line of code) (Count the total no. of lines written)
(2) FPA (Function point Analysis) (Measures the task the user expects from the software)
(3) UCP (Use case points) (Based on use case diagram).

(B) Size Estimation technique :-

(1) Expert judgement
→ Estimation based on the experience of experts.

(2) Delphi technique
→ Estimate from several different experts and calculate their average.

(3) Analogy-based estimation
→ Estimate using the last completed similar project as a reference.

(4) Top-down estimation
→ first estimate the whole system, then break into the smaller parts.

(5) Bottom-up estimation
→ opposite to top-down.

Cost estimation in Software project :-

Types of Cost estimation :-

(A) ROM (Rough order magnitude) [Initial estimate]
(B) Budget estimate [used for planning]
(C) Definitive estimate [when complete information about the project available. a definitive estimate is made]

* Factor affecting Cost :-

→ Size of the project.
→ Complexity of the project
→ Team experience
→ Technology used
→ Onsite & offshore
→ Deadline.

* Cost estimation Techniques:-

(1) Algorithmic/Parametic methods.
(a) Cocomo model (b) FPA.
(2) Empirical method :-
(a) Exper Judgement (b) Delphi technique
(3) Analogous Estimate
Use data from previously completed similar projects.
(4) Top-down
first whole project then smaller part
(5) Bottom-approach.
opposite → Top-bottom

* Staffing level Estimation :- (work force).

(A) Create WBS (work breakdown structure)
→ Divide the project into small parts.
→ fix the required time and staff for each part.

(B) Effort estimation :-
→ find how many hrs are needed → complete each task.

→

* DFD (Data flow diagram) :-

→ A graphical tool that shows as a single process.
→ ~~clearly represents 4 things :-~~
→ known as Bubble chart.

* DFD components :-

(1) External entity :-

→ People and organizations that interact with the system.
→ Represented by a Rectangle.
→ They send data to the system or receive from it.
ex: Bank, customer etc.

(2) Process :-

→ The work of processing data in the system.
→ Circle / ellipse
→ ex: Processing an order, validating data.

(3) Data store :-

→ A place where data is store temporary or permanently
→ file, database
→ open-ended rectangle (=).

(4) Data flow :-

→ used to show how and from where & where data moves inside the system.
→ arrow : →
→ Show direction of data.

* Types of DFD :-

(1) level 0/context level DFD :-
→ Highest level view of the system.
→ The whole system is shown as a single process.
→ only external entities and main data flow are shown.

(2) level 1 DFD :-
→ The main process is divided into sub-process.
→ Shows how data flows b/w these sub-processes.

(3) level 2,3 (Detailed DFD) :-
→ Level 1 explained in more detail.
→ used to show the system in greater depth.

Rules of DFD :-

→ Every process must have at least one input and one output.
→ Data cannot go directly from one data store to another data store.
→ There is no data flow directly between external entities.
→ one process should not communicate directly with another process.
→ while data is being processed, its flow is not shown in two opposite directions

* Object oriented Design:-

Characteristic:

1) class (Template used to create obj)
2) object (an instance of class)
3) Encapsulation (Keep data secure from external access)
4) Inheritance (Uses the properties of one class in another class)
5) Abstraction (Hide complex details, shows the required one)
6) Polymorphism (using fn with same name in different ways)

* Principle of OO Design:- SOLID

(i) S = SRP (Single Responsibility Principle)

→ Every class should have one responsibility/one reason to change.

(ii) O = open/closed Principle

→ Software entities should be open for extension but closed for modification.

(iii) L = Liskov Substitution Principle.

→ Subclass should be usable in place of their superclass.

(iv) I = Interface Segregation principle.

→ No client should be forced to depend on interfaces it does not use

(v) D = Dependency Inversion Principle.

→ Clients should depend only on abstractions, not on concrete implementations.

* SRQA → Software Reliability & Quality Assurance.

Software Quality (whether it fulfills user requirement or not)

Characteristics:

(a) Functionality
(b) Reliability
(c) Usability
(d) Efficiency
(e) Portability
(f) Maintainability
(g) Correctness

→ Quality Assurance is a process that ensures whether a Software product is following predefined standard or not.

→ MTBF (Mean Time Between Failure) denotes the average time for which the Software keeps working without failing.

→ Quality Control is a technical process that helps the Software to determine its Quality.

→ For Software Quality Standard, ISO 9001, CMMI (Capability maturity model Integration) for process improvement. and Six Sigma for reducing errors/risks are used as Standards.

* Musa's Basic Model :-

→ This is a model of instructional technology, created by Musa.
→ It is used in the field of education → to make effective communication and the teaching-learning process better.
→ Four Components :-
- Source → message → channel → Receiver.

Purpose :
→ To make communication effective
→ To make ~~teaching~~ techniques easy and simple.
→ To help teachers understand how information reaches students.

* ISO 9000 Series :-

→ International organisation for Standardization
→ ISO 9000 QMS (Quality management System)
→ main characteristics :-
(a) Customer focus
(b) Leadership
(c) Engagement of people
(d) Process approach
(e) Continuous improvement.
(f) Relationship management.
(g) Evidence based decision making.

* CMM :-

→ SEI-CMM (Software Engineering Institute Capability maturity model)

→ Use for process maturity in software organisation.

Level of CMM :-

(A) Level 1 (Initial level) :-

→ No fix process

→ No fix manners

(B) Level-2 (Repeatable) :-

→ Repeat the success process.

(C) Level-3 (Defined) :-

→ Process documentation defined.

(D) Level-4 (managed) :-

→ Process & Quality management.

(E) level-5 (Optimizing) :-

→ Regular improvement.

Features	Iso 9000	SEI CMM
1) Type	QMS (Quality management System)	Process maturity Model
2) Focus	Quality Assurance & Customer Satisfaction	Software process improvement
3) Certificate	By ISO	No. Certification.
4) Process Approach	Requirement of Process	Maturity of process

* Software maintence :-

- Timely correction and modification. (work smoothly)

Types of maintence :-

(A) Corrective :-

→ corrective bugs and errors

→ Ex:- Runtime and logical err.

(B) Adaptive :-

→ making any software capable of running on new hard. or os.

c) Perfective :-

→ making a new version of old software available, more better.

d) Preventive :-

→ update made available for security against future problem and errors

* Software maintence Process model:-

(a) Quick fix model:-

→ Centralized on directly Solving the problem.

(b) Iterative Enhancement model:-

→ Slowly-slowly making Correction in the Current System.

(c) Reuse Orientation model:-

→ Reusing available Code & Content.

(d) Boehm's model / SMW (Software maintence wheel):-

→ Circular model which includes identification of change, impact analysis, modification, testing & release

* Reverse Engineering:-

→ It is a process in which we fetch design, and documentation of a currently available software without changing its code.

3 steps:-

(i) Information Extraction (Source code & DB Schema)

(ii) Abstraction (Covering from low level to high level design)

(iii) Documentation